

Software Design Description

Francanglais Compiler and Dataset Platform

Group members and matricules:

Name 1 — Matricule 1

Name 2 — Matricule 2

Name 3 — Matricule 3

Version 1.0 — September 19, 2026

Group work document. Replace all red placeholders before submission.

Contents

1	Design Basis and Evidence	3
2	Architectural Overview	3
3	Deployment View	4
4	Detailed Module Design	5
4.1	App	5
4.2	Dataset	5
4.3	Audio utilities	6
4.4	Compiler boundary	6
4.5	Class relationships and multiplicities	6
5	Behavioral Design	9
5.1	Use cases	9
5.2	Entry collection activity	10
5.3	Save sequence	10
5.4	Entry lifecycle	13
6	Interfaces and Contracts	13
7	Persistence and Error Handling	14
8	Security and Privacy Design	14
9	Verification Strategy	14
10	Design Decisions and Future Work	14

1 Design Basis and Evidence

The design is grounded in the source-controlled implementation: `data_collector/App.py`, `dataset.py`, and `audio_utils.py`. The collector uses a CustomTkinter tabbed interface, CSV persistence, and a managed audio directory. The repository also includes a compiled frontend bundle and local ignored backend/compiler artifacts; those are represented as integration boundaries, not as verified public APIs.

2 Architectural Overview

The system follows a local layered design:

1. **Presentation layer:** App builds Collect, Browse & Edit, and Stats tabs and handles user commands.
2. **Dataset layer:** `dataset.py` owns CSV initialization, reading, append, replacement, update, deletion, duplicate checks, and counts.
3. **Audio layer:** `audio_utils.py` records, copies, saves, and plays audio with graceful capability detection.
4. **Compiler boundary:** a future lexer/classifier/parser consumes dataset evidence without coupling the UI directly to parsing internals.

Francanglais Platform - Component Diagram

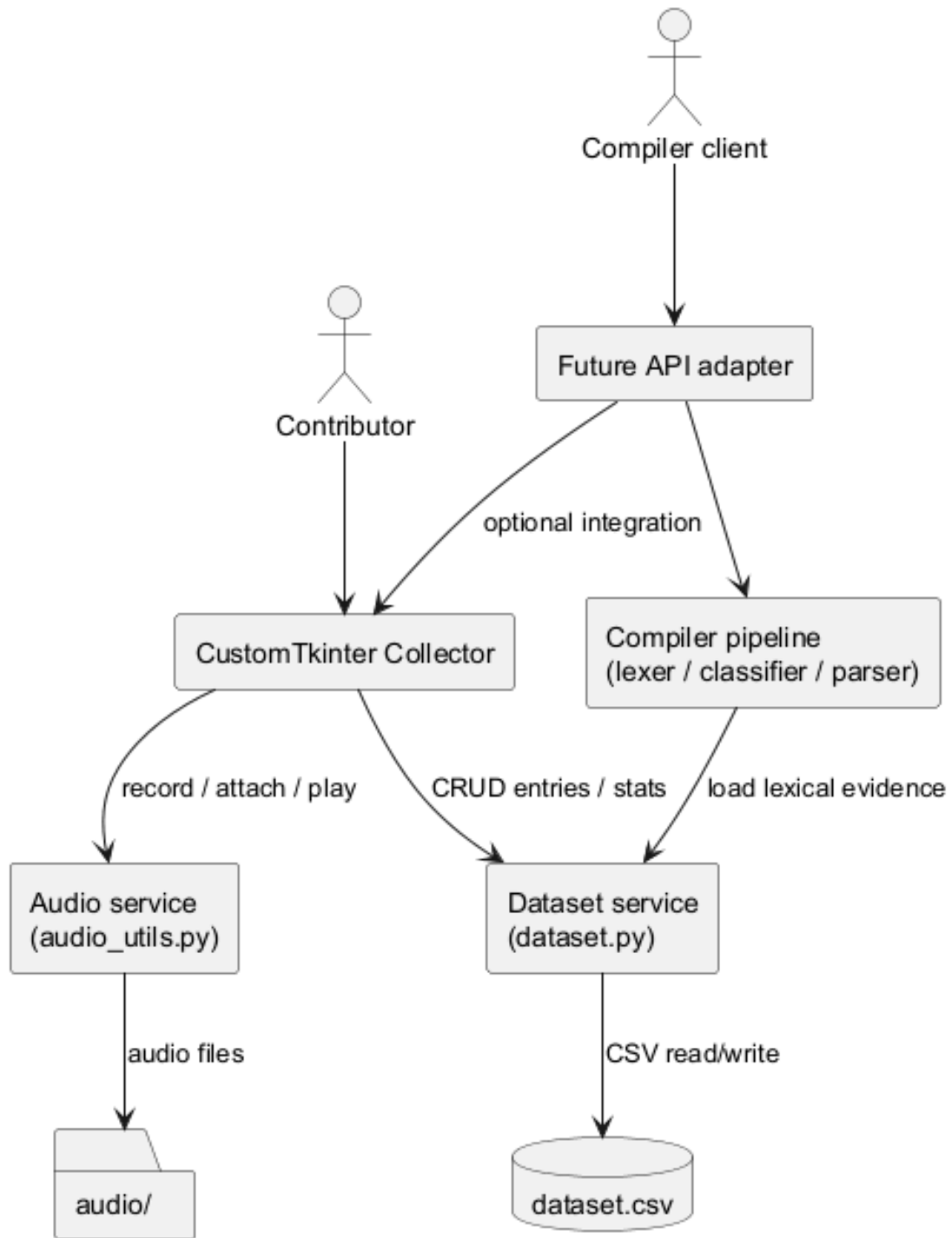


Figure 1: component

3 Deployment View

The implemented collector is deployed on one contributor workstation with Python runtime, local CSV storage, and local audio files. A compiler service or API adapter is optional and is shown separately to prevent confusion between current and planned deployment.

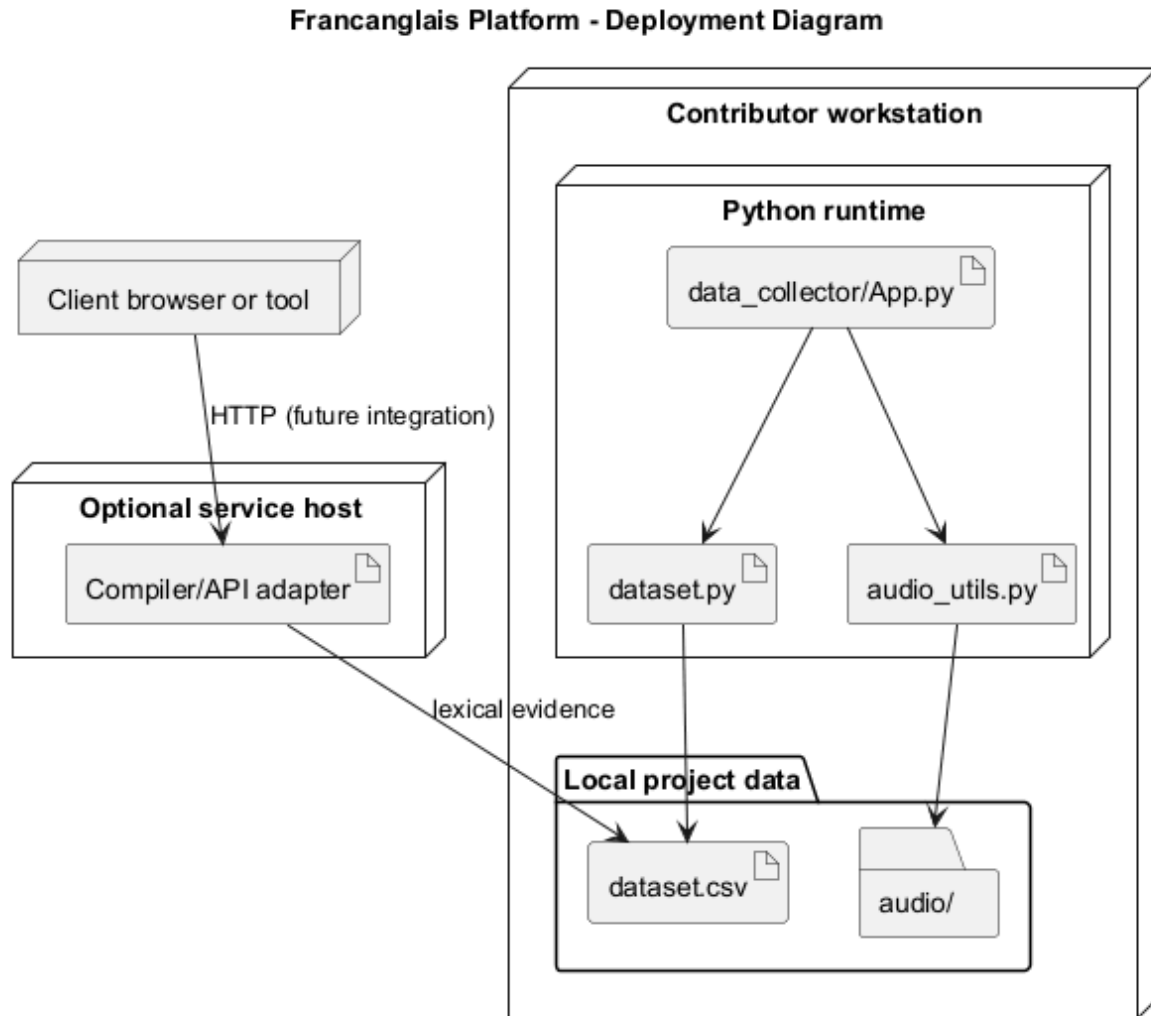


Figure 2: deployment

4 Detailed Module Design

4.1 App

App initializes the dataset file, configures the dark UI, creates three tabs, and owns transient form state such as the pending audio filename and selected browse row. `save_entry` reads the form, creates an ID and timestamp, calls `dataset.append_entry`, and refreshes visible state. Browse operations delegate persistence to dataset helpers. Audio controls delegate recording and playback to `audio_utils`.

4.2 Dataset

The dataset module defines the canonical field list, entry types, and categories. `ensure_dataset_file` establishes the CSV header; `load_all` returns dictionaries; `append_entry` appends one row; `save_all` rewrites the complete file for updates/deletes. `text_exists` normalizes case and whitespace for duplicate detection.

4.3 Audio utilities

`AUDIO_RECORDING_AVAILABLE` is determined by optional imports. `Recorder` buffers microphone frames and returns one NumPy array. `save_recording` writes a UUID-named WAV file. `attach_file` copies an existing file to a UUID-named destination. `play_audio` tries `sounddevice/soundfile` first and then the operating-system default player; it returns false when the path is missing or both paths fail.

4.4 Compiler boundary

The compiler design is intentionally staged: tokenize text, classify tokens using exact/fuzzy and learned lexical evidence, parse a mixed-language sentence with a grammar, and return a structured analysis. This boundary should be implemented behind an adapter so the collector remains usable without the compiler and so compiler analysis cannot silently rewrite data.

4.5 Class relationships and multiplicities

The class diagram includes both application-defined classes, `App` and `audio_utils.Recorder`, together with the relevant external classes `customtkinter.CTk` and `sounddevice.InputStream`. It shows selected attributes and methods rather than every GUI widget. `App` inherits from `CTk` and owns the optional recorder; the audio helper module does not own a recorder instance.

The `module`, `record`, and `logical storage` stereotypes distinguish supporting design elements from implemented classes. In particular, `dataset.py` and `audio_utils.py` are modules whose functions are shown as static operations. `DatasetEntry` describes the CSV row/Python dictionary schema; `DatasetFile` and `AudioFile` describe stored files. These three names are not Python model classes.

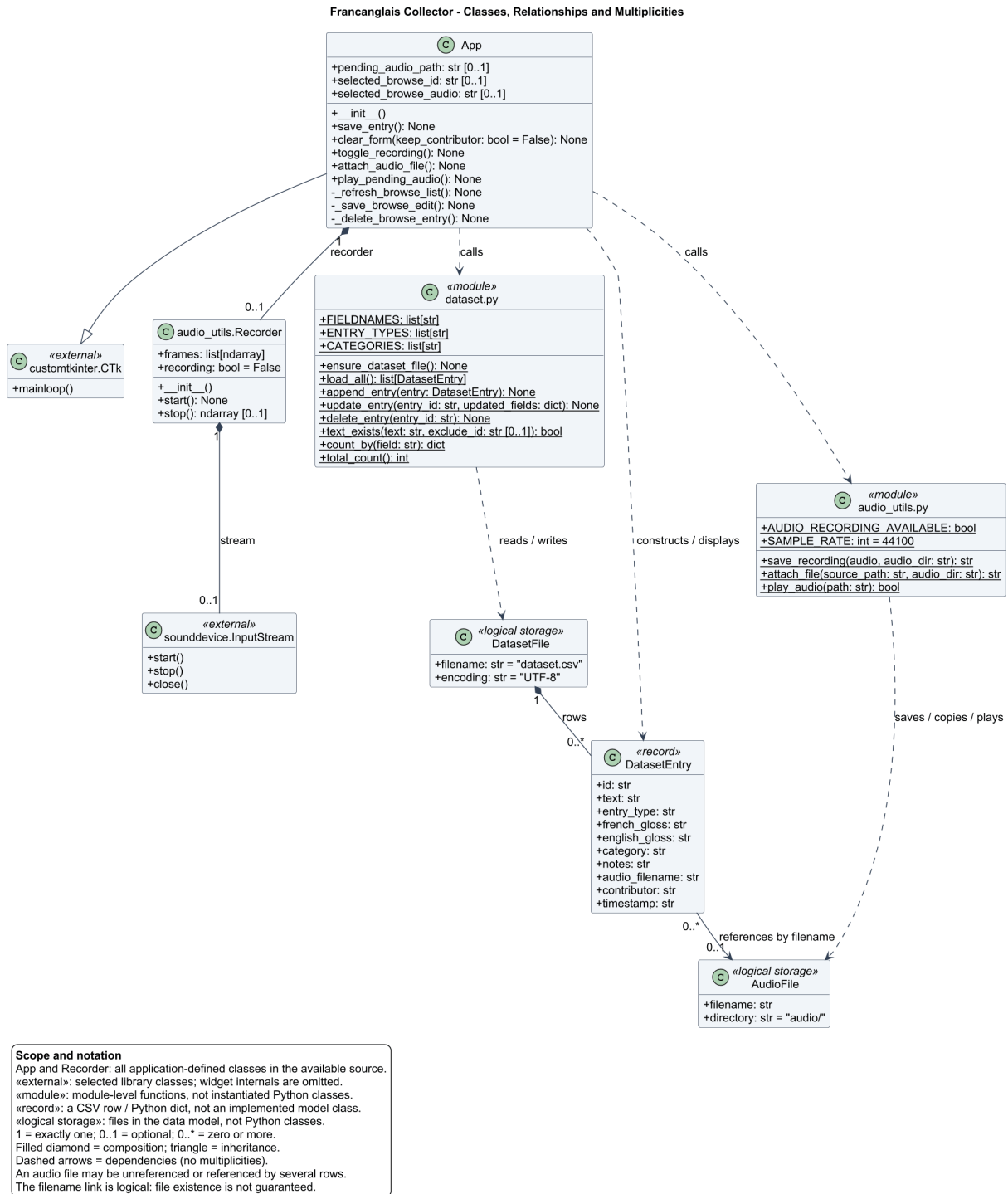


Figure 3: class-domain

Relationship	Multiplicity and meaning
App–Recorder	One app owns zero or one recorder, depending on audio capability. Each recorder created by the collector belongs to one app.
Recorder–InputStream	One recorder owns zero or one stream. The stream is created when recording starts and closed and released when it stops.

Relationship	Multiplicity and meaning
DatasetFile– DatasetEntry	One CSV file contains zero or more rows; each stored row belongs to one CSV file in this data model.
DatasetEntry–AudioFile	A row optionally references one audio file by filename. A file may have zero or more references because filenames are not constrained to be unique across rows.

Filled diamonds denote composition and sit at the owning (whole) end. The audio-file link is an association, not composition: deleting a CSV row does not delete its audio file. This logical link does not guarantee that the referenced file still exists. Inheritance and function-call dependencies do not have multiplicities; adding cardinalities to those arrows would not be valid UML.

5 Behavioral Design

5.1 Use cases

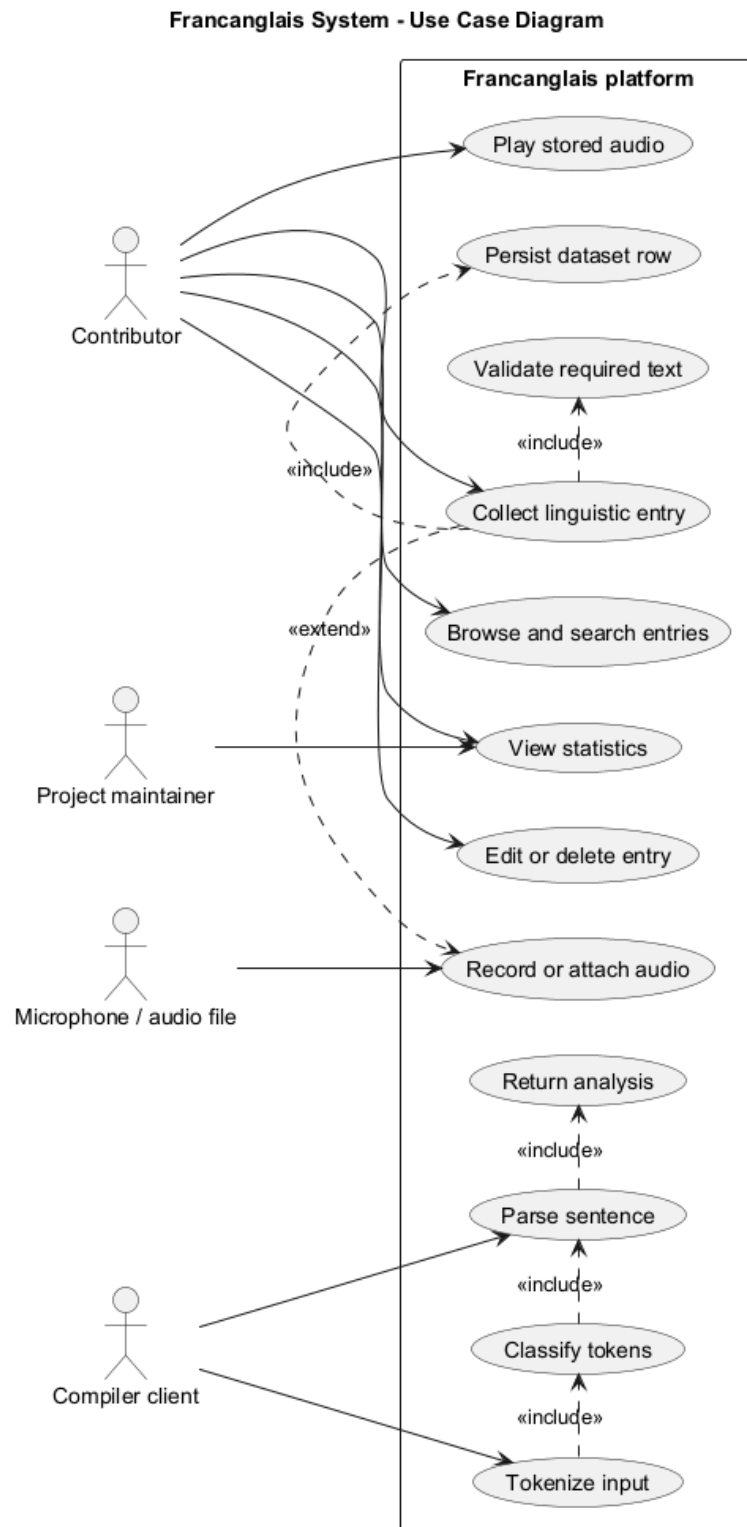


Figure 4: use-case

5.2 Entry collection activity

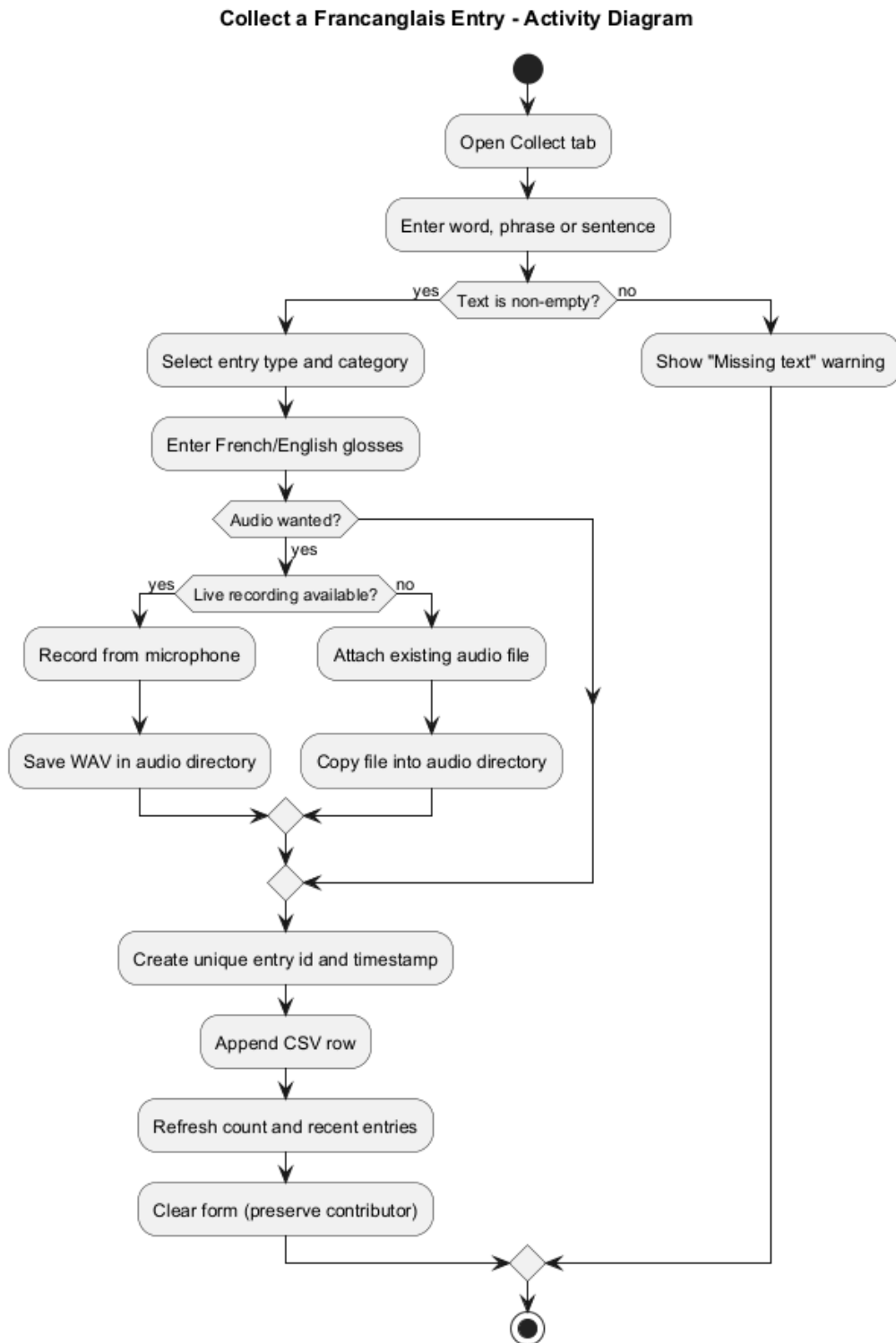


Figure 5: activity-collect

5.3 Save sequence

The sequence shows an optional existing-file attachment before saving, followed by mutually exclusive empty-text and valid-text paths. An empty submission returns a warning without

a CSV write. On the valid path, the application persists the entry, refreshes counts and recent entries, and clears the form while retaining the contributor.

Every synchronous message starts an activation on its receiving lifeline, and every dashed reply ends that activation. A self-call creates a nested activation; the caller remains active while waiting. The contributor's activation spans each user interaction. Module lifelines represent executing functions, and the file-I/O lifeline abstracts local filesystem operations, not an additional application class or remote service. Individual widget calls and lower-level file helpers are omitted. Live recording and storage exceptions are outside this particular scenario.

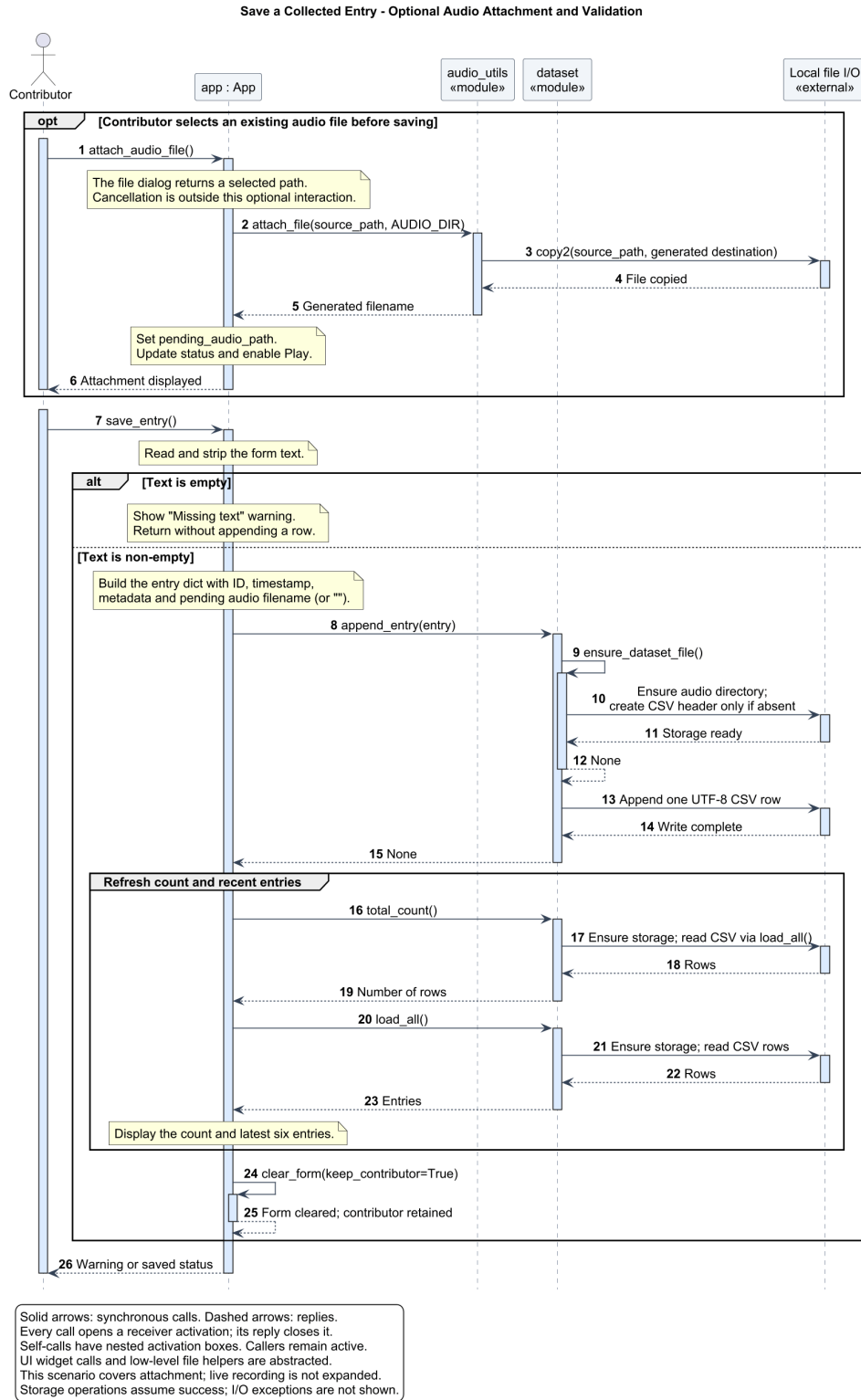


Figure 6: sequence-collect

5.4 Entry lifecycle

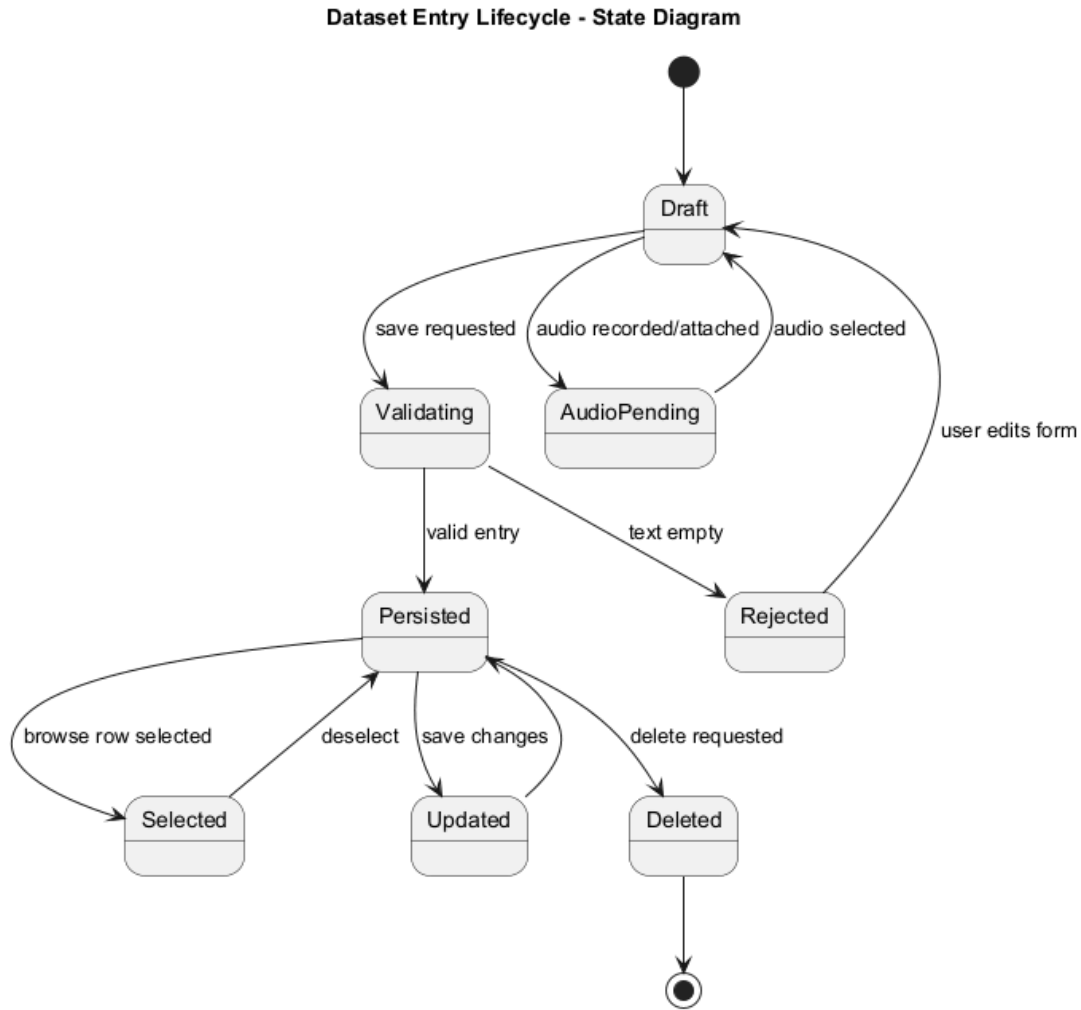


Figure 7: state-entry

6 Interfaces and Contracts

Interface	Contract
<code>dataset.append_entry(entry)</code>	Receives a dictionary containing the canonical fields and appends one UTF-8 CSV row.
<code>dataset.update_entry(id,fields)</code>	Loads rows, updates the matching ID, and rewrites the CSV.
<code>dataset.delete_entry(id)</code>	Loads rows, removes matching ID, and rewrites the CSV.
<code>audio_utils.attach_file(source,dir)</code>	Copies a source media file to the managed directory and returns its generated filename.
<code>audio_utils.play_audio(path)</code>	Returns true only when playback was successfully started.

Interface	Contract
<code>CompilerAdapter.analyse(text)</code>	Proposed contract returning tokens, token labels, parse status, and explanatory diagnostics.

7 Persistence and Error Handling

The CSV header is created before reads or writes. Update and delete use read-modify-write semantics, which is suitable for a single desktop user but not for concurrent writers. Empty text is rejected at the UI boundary. Missing audio paths and playback failures are surfaced as false results and, for pending playback, a visible error dialog. Optional recording import failure disables recording while preserving attachment and data-management features.

8 Security and Privacy Design

Local environment files may contain credentials and are excluded from the documentation and Git history. Audio and linguistic entries can contain personal or culturally sensitive information; access should be restricted to the project team, contributor consent should be obtained, and exports should exclude secrets. A future service adapter must validate input, authenticate maintainer operations, and avoid logging raw audio or credentials.

9 Verification Strategy

- Unit-test CSV initialization, UTF-8 round trips, append, update, delete, duplicate normalization, counts, and empty-input validation.
- Test audio capability-off behavior without a microphone, plus attach and missing-file playback paths.
- Test compiler tokenization and parsing against representative French/English/Pidgin mixtures and malformed input.
- Perform an end-to-end manual scenario covering Collect, Browse & Edit, Stats, and audio playback.
- Regenerate diagrams from the PlantUML sources so the PNGs remain consistent with the design text.

10 Design Decisions and Future Work

- CSV is retained for transparent, portable Phase 1 collection.
- The compiler remains decoupled until the corpus and grammar are stable.
- A database, schema migration strategy, authentication, and service API are future work for multi-user deployment.
- The illustrative grammar in the coursework JSON requires linguistic validation before being promoted to a production parser.